

Install the Postman MCP Server in VS Code

For this install I am using an older Ubuntu 24.04 and Node version 24.14.0

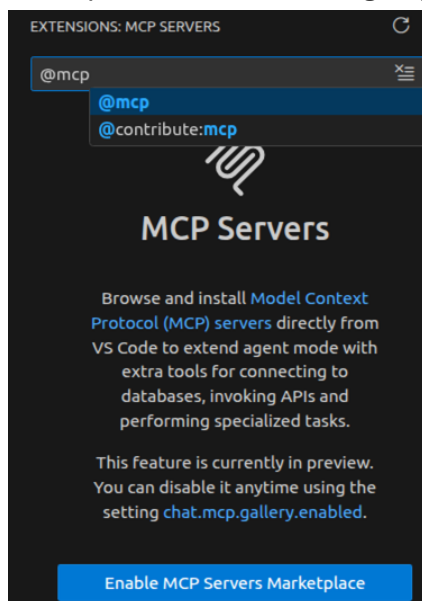
Before you begin this process, create an account and sign in to that account at Postman.com. You may be able to sign in to Postman from VS Code. You must have `npx` installed in your Linux system before proceeding.

If you are working with this document after installing GitHub MCP Server or MongoDB MCP Server, some of these steps will be redundant. You will have to skip a few of them. You will be editing the same `mcp.json` file from the GitHub/MongoDB install, do not create a new one.

Unless Postman has changed their policy, this part of the bootcamp involves obtaining an API key from the Postman website. Please keep this key private.

Part 1 – Extension Installation

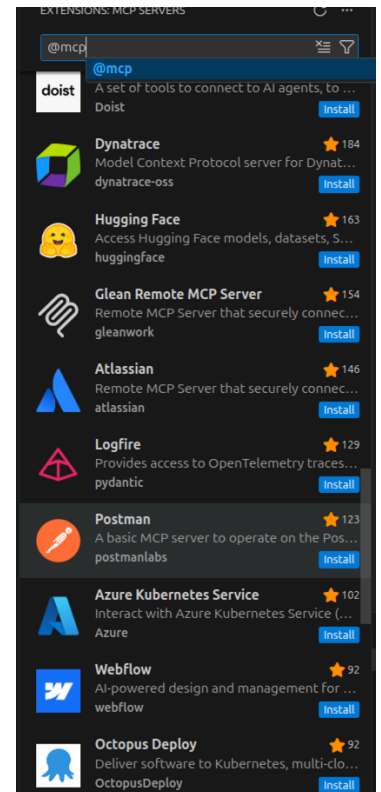
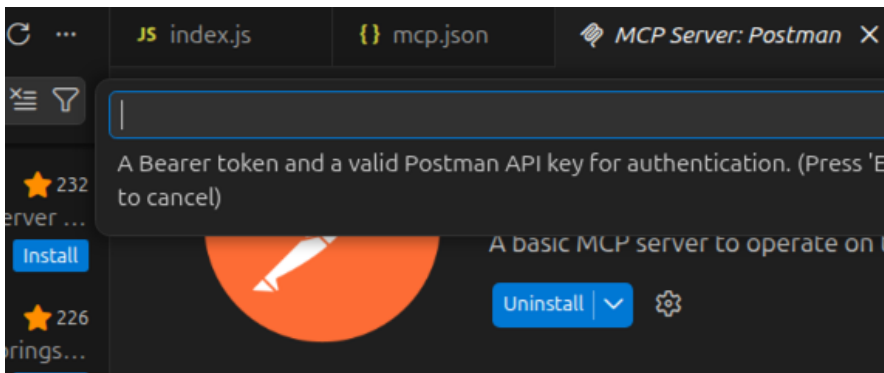
1. Open the Extensions panel (left of main editor window). You will see a search bar at the top of that window, begin typing `@mcp`. This will trigger the MCP list to show up.



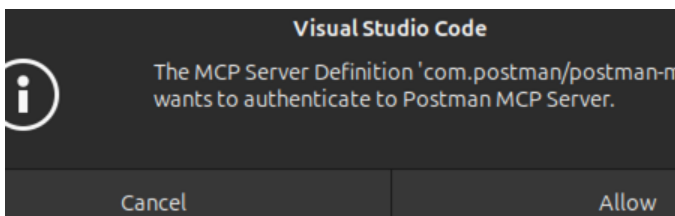
2. Choose Postman and then click the blue Install button. You may be asked to sign in, if you have not already done so.

The next three options will appear depending on your own setup.

3. Click on the `Enable MCP Servers Marketplace`
4. You will get a second popup that you can click `Enable` on
5. Once you have done this, you will see a list of all MCP Servers available at the current point in time. In my case, Postman is further down the list. *This list changes with time.*
6. Select `Postman` and use the `Install` button on the main window to get this server installed on VS Code (see #2)
7. Again, depending on your configuration, you may be asked for permission to view Postman's collections:



Click **Esc** here. You may then be asked to authenticate the server, click `Allow`:



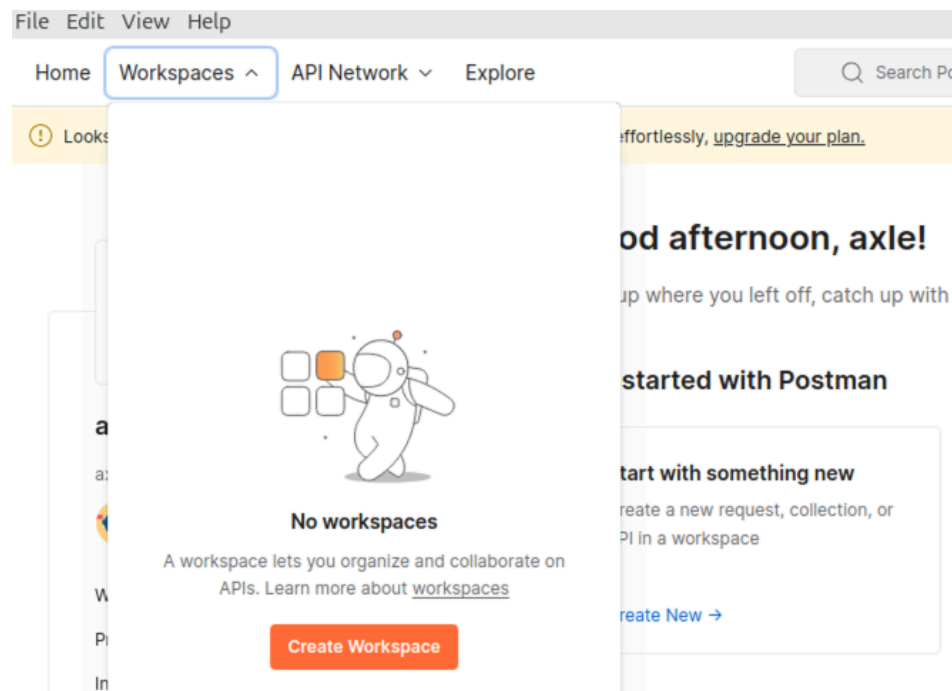
From there you will have to authenticate with the online Postman site.

Part 2 – Permissions

Note: some of the procedures/screens/permissions may have changed by the time you are taking this bootcamp. Just follow the instructions of the service at the point in time.

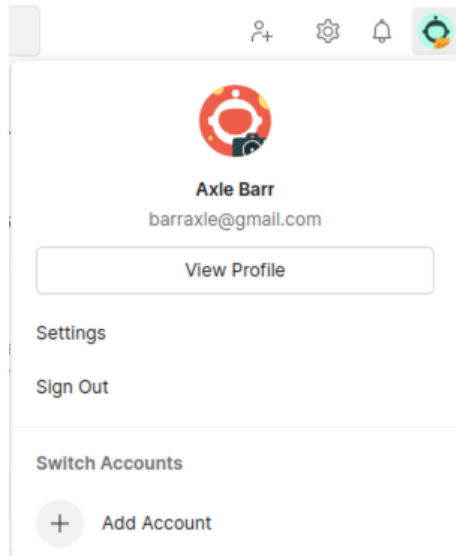
1. If you have Postman installed, open it and sign in. If you do not have a Postman account you can sign in with your regular Gmail account. (They may have other options by the time you are reading this document).

The image below confirms that I do not have any workspaces or collections as yet. See below #2. This image is from my local Postman app in Ubuntu 24.



With Postman, you can work on the remote postman.com or local Postman application.

2. Once you sign in on your local Postman app, go to the account settings:

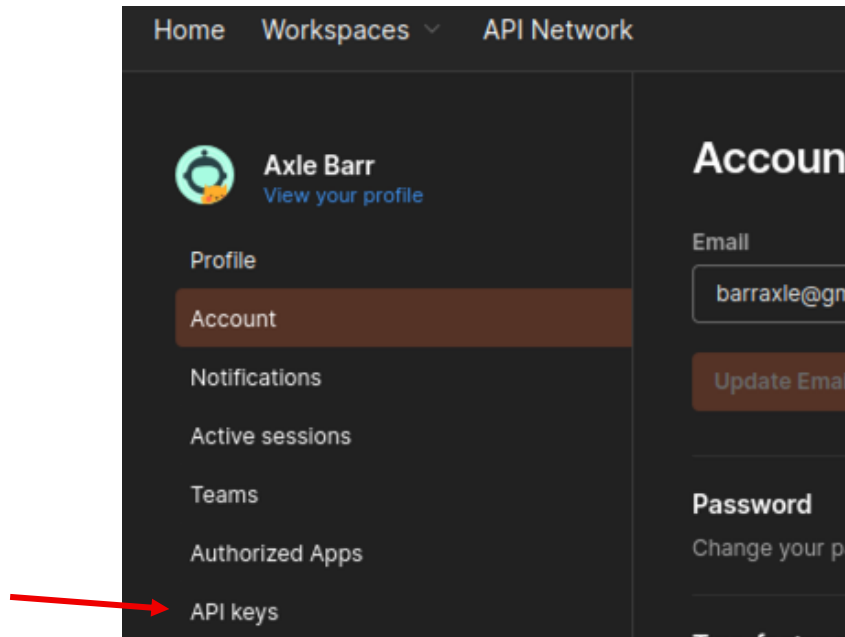


Postman likes to create workspaces and collections.

A workspace is a shared space or container for organizing your overall work and collaborating with teammates, while a collection is a grouping of individual API requests that you can reuse, automate, and share.

At the time of this document, you must create a workspace and then a collection if you want to work with the MCP Server. You will also need an API Key, see #3 below. If you are just now starting to work with Postman, your account will be empty, no workspaces or collections. As you proceed through the bootcamp, these may be created for you automatically.

3. From the image above, click on **Settings** and it will open up a web interface where you have access to API keys

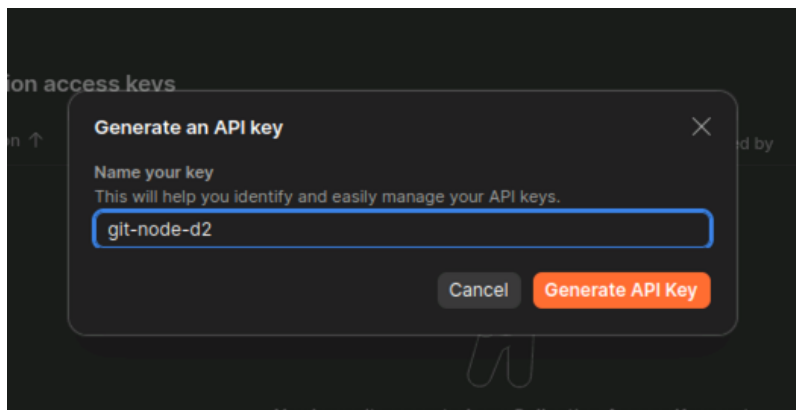


Once you click on the **API Keys** link, you will have an opportunity to **Generate API Key** using the orange button on the top right of that UI.

You will have to give it a name and the key will appear. Copy it to somewhere safe, we will use it in the next step.

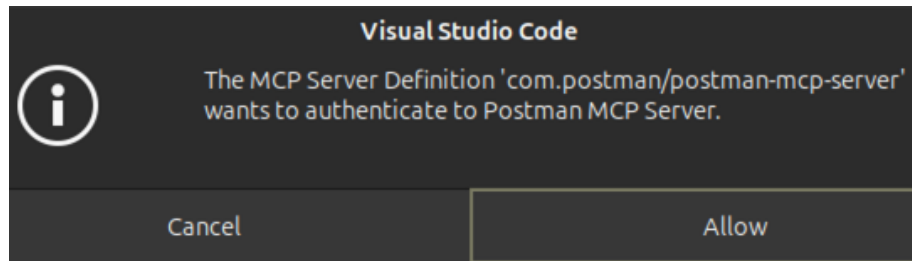
Note, if you have not used your API key in some time, you may be asked to regenerate a fresh one.

4. This is me generating an API key for the current bootcamp project:



Note the name of the workspace, if you do not have a workspace, you may not be able to generate a key. You can always come back to this step after creating one.

5. Once you have your key, it is time to enter it into the input prompt box in VS Code. See part 1 #7. VS Code will then ask:

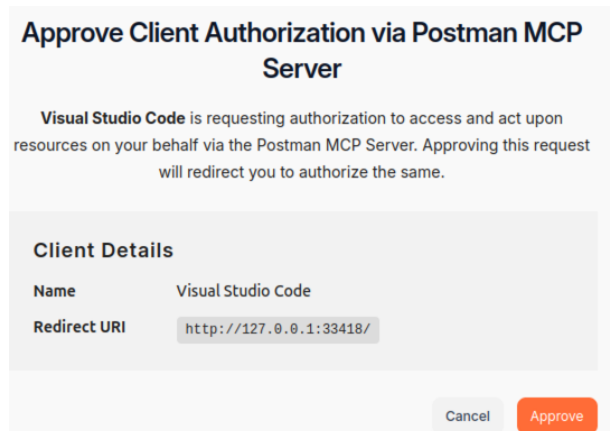


If you missed this opportunity to enter your key, you could always enter it directly into the `mcp.json` file or have it read from this file via a `.env` file (Part 3)

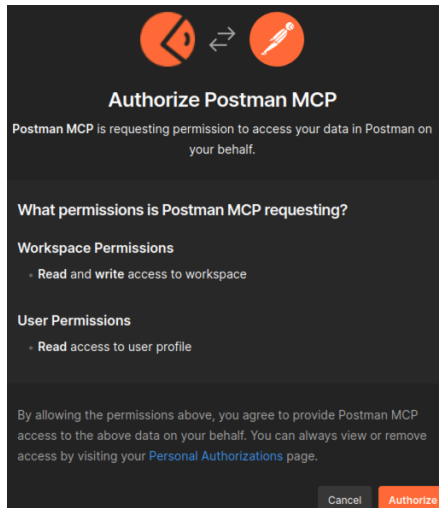
Note, the following tasks may have changed by the time you are participating in the bootcamp.

If you got caught in these steps, just continue, otherwise we will create a `.env` file and add our key there for future use.

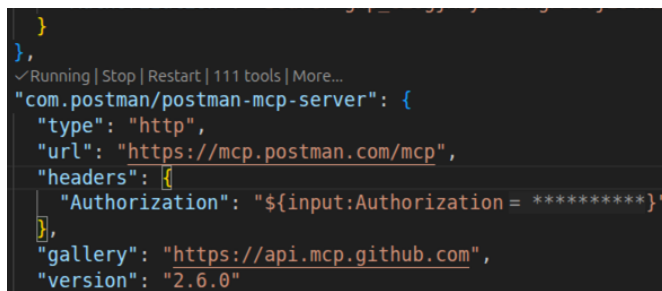
6. It will then ask you to open an external website
7. Then it will take you to that site where you can Approve the request:



8. It will then ask for specific permissions:



9. Back in VS Code, your `mcp.json` file should now be updated to reflect Postman:



10. If you installed the GitHub MCP Server and MongoDB MCP Server, you will now have three MCP Servers installed. Notice how they each have their own start/stop link in the `mcp.json` file:

```
{
  "servers": {
    >Start | 37 tools | 2 prompts | More...
    "io.github.github/github-mcp-server": {
      "type": "http",
      "url": "https://api.githubcopilot.co
      "gallery": "https://api.mcp.github.c
      "version": "0.30.3",
      "headers": {
        "Authorization": "Bearer ghp_8Zogj
      }
    },
    ✓Running | Stop | Restart | 111 tools | More...
    "com.postman/postman-mcp-server": {
      "type": "http",
      "url": "https://mcp.postman.com/mcp"
      "headers": {
        "Authorization": "${input:Authoriz
```

11. We can now submit a query to see what happens. The image below shows the prompt and part of the result list:

```
#com.postman/postman-mcp-server
show me all my collections

✓ Optimizing tool selection...

I'll hook into the Postman MCP collection-
management tools so I can query your
account and list all your collections. Now I'll
call Postman's list-collections API to fetch all
collections in your chosen workspace.

Ran The workspace ID query is required for
this endpoint. If not provided, the LLM
✓ should ask the user to provide it. -
com.postman/postman-mcp-
server (MCP Server)

Here are your Postman collections from the
current workspace:

• Local Environment — id: 68ebce61-3974-
4631-a4de-489643d8a4dd, uid: 6447137-
68ebce61-3974-4631-a4de-
489643d8a4dd
• Local Environment — id: b4050fad-275a-
45a2-9316-77b6a169c111, uid: 6447137-
b4050fad-275a-45a2-9316-
77b6a169c111
```

Part 3 – The Environment File

Since the Postman MCP Server works with an API key, it is best to have this key read by your app and not be exposed directly in the `mcp.json` file. The key can be loaded automatically from a file or injected from your system's environment. We will go with the first option in this case.

1. Create a `.env` file directly in the root of your application and add this line of code:

```
# .env
POSTMAN_API_KEY=PMak-65fb...your-full-key-here...
```

Remember that all files that start with the period is considered a hidden file.

2. Also enter the connection string for the MongoDB database:

```
# .env
POSTMAN_API_KEY=PMak-65fb...your-full-key-here...
MDB_MCP_CONNECTION_STRING=mongodb://localhost:27017
MDB_MCP_READ_ONLY=false
```

3. Open the `mcp.json` file, stop the MongoDB MCP Server and prompt Chat to adjust the file to read the connection string for MongoDB from the `.env` file:

```
I have created a .env file and added my MongoDB database
connection string there, update the app to get the connection
string from that file
```

There is a special bash command to run for this, let the AI guide you.

4. Run a similar prompt for the Postman API key:

```
I have created a .env file and added my Postman API key
there, update the mcp.json file to get the key from that file
```

5. Now just update the `mcp.json` file to reflect the change from #2:

```
"args": [
  "-y",
  "@postman/postman-mcp-server",
  "--minimal"
],
"envFile": "${workspaceFolder}/.env"
},
```

If you do not have the `args` key, use the code in #6 below

6. Update the `.gitignore` file so that the API key is not uploaded and become exposed:

```
node_modules
.env
```

You should already have the `node_modules` folder in your `.gitignore` file

7. This is just a part of the `mcp.json` file after all the changes. Notice that there is now a bash command added so that the env file can be read:

```
{
  "mongodb": {
    "type": "stdio",
    "command": "bash",
    "args": [
      "-lc",
      "set -a && source /home/axle/Documents/git-node-d2/.env && exec npx -y mongodb-mcp-server"
    ]
  },
  "postman": {
    "type": "stdio",
    "command": "bash",
    "args": [
      "-lc",
      "set -a && source /home/axle/Documents/git-node-d2/.env && exec npx -y @postman/postman-mcp-server --minimal"
    ]
  },
  "io.github.github/github-mcp-server": {
    "type": "http",
```

You may be asked to restart all servers. I usually close the current folder in VS Code and reopen it. This ensures that all changes were applied.

8. At this point, just ask Copilot to test that the API key is being read correctly:

```
Verify that the Postman API key is being read from the .env file
```

This may not work at this point, complete #7 first.

9. While we are here, you can move your MongoDB connection string to the `.env` file

Move the mongodb connection string to the `.env` file and adjust the `mcp.json` file for this move

This may already have been done.

10. This is what my `.env` file looks like now:

```
.env
1 # .env
2 POSTMAN_API_KEY=PMAC-6957e96649
3 MDB_MCP_CONNECTION_STRING=mongo
4 MDB_MCP_READ_ONLY=false
5
```

Of course I will not show the actual values

11. This is just a part of the `mcp.json` file after all the changes. Notice that there is now a bash command added so that the env file can be read:

```
{
  "mongodb": {
    "type": "stdio",
    "command": "bash",
    "args": [
      "-lc",
      "set -a && source /home/axle/Documents/git-node-d2/.env && exec npx -y mongodb-mcp-server"
    ]
  },
  "postman": {
    "type": "stdio",
    "command": "bash",
    "args": [
      "-lc",
      "set -a && source /home/axle/Documents/git-node-d2/.env && exec npx -y @postman/postman-mcp-server --minimal"
    ]
  },
  "io.github.github/github-mcp-server": {
    "type": "http",

```

Part 4 – Postman Local Environment Setup

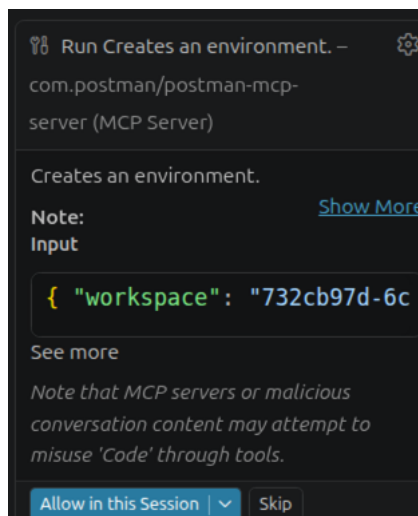
Before we continue, if you got lost along the way, there is a complete application, up to this point in the zipped folder for today. Look for **git-node-d2-p3.zip**. This file will contain the entire application up to this point. Only two of the four routes will work, the root and /aboutus routes will work. We will work on the POST route in this part. Also in this zipped file there is a **.env** file. It will contain the three lines of code needed for the app to function; however you will need to insert your own Postman API key.

1. Create a Postman Environment (e.g., named "Local Dev") and set your baseUrl to your localhost address:

```
Using my Postman MCP Server, help me setup a personal workspace named "bootcamp" and local environment named git-github, based on the routes in this file of this NodeJS application. I am using localhost port 8000.
```

You can open the [routes.js](#) file for context, but not necessary

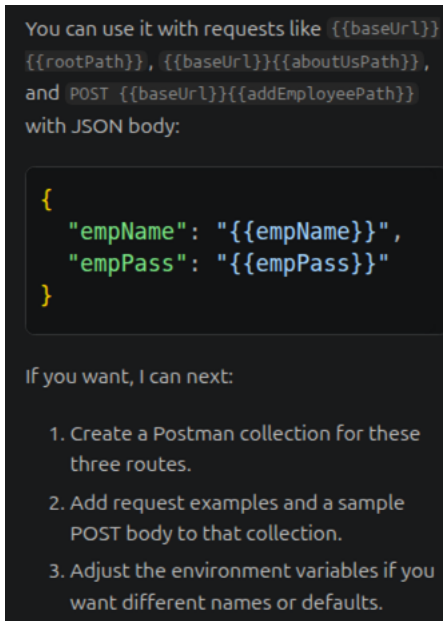
This is MY response below, yours may be different:



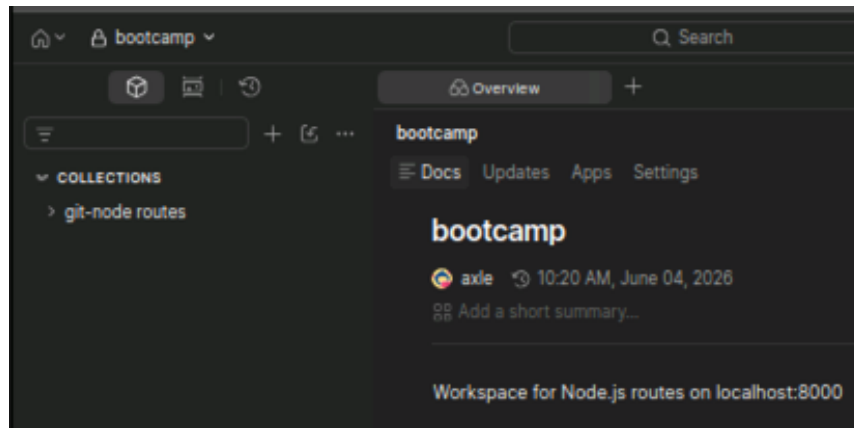
Here is the written response:

I created a Postman environment named git-node-d2 Local in your Personal Workspace.

After I clicked Allow... this is the response:



2. If the prompt above was successful, go to your Postman.com site and the workspace will appear there:



You may have to refresh the browser.

3. Since the MCP server works best when it has a structured "map" of the API to look at, letting the AI generate the collection saves manual labor and ensures the variables (like `{{baseUrl}}`) match what the AI expects. Here is my response from the last image from #1:

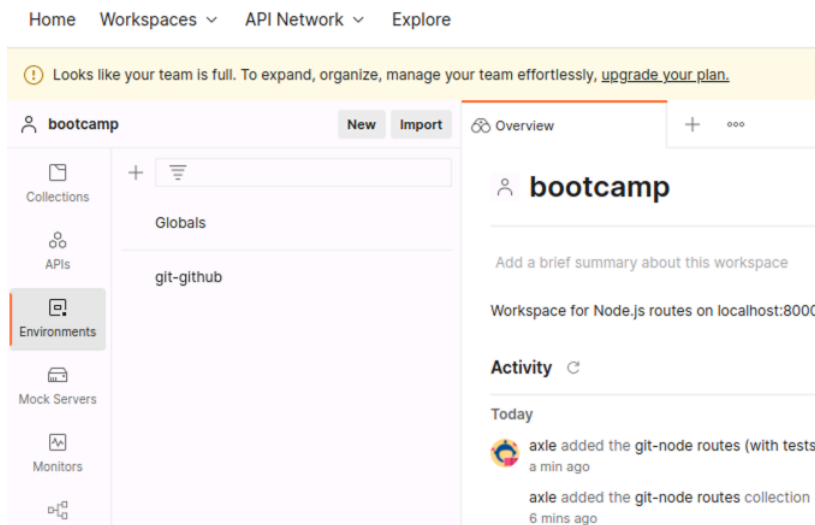
Proceed with all three bullet points

4. After completing all the tasks from #3, this is the response, however I will not go further at this point. I want to check what Copilot has done for me so far:

```
If you want to keep going, the natural next steps are:
```

1. I can create a monitor or mock for this collection.
2. I can update your Express route so `POST /addemployee` actually persists an employee with the schema in `employee.js:1`.
3. I can add a Postman environment or collection for a second stage such as `dev` vs `local`.

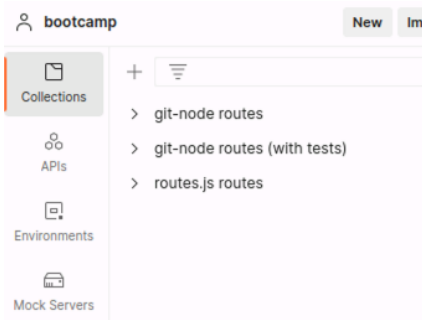
5. Open your local Postman application and verify that the environment and collection was setup correctly. This is my local Postman application in Linux. This is after #4:



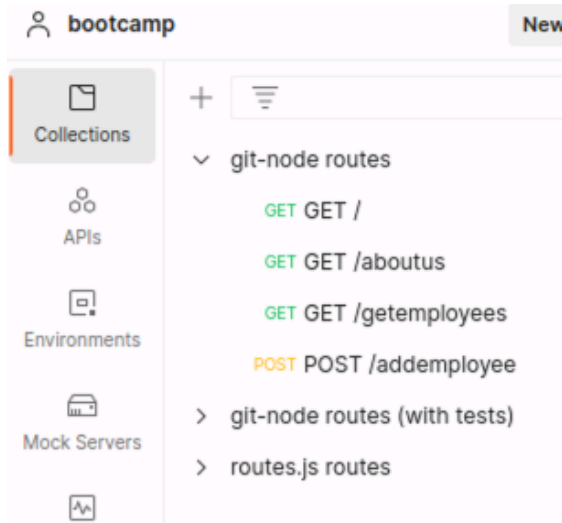
6. Now that I have confirmed the environment, I can proceed to add a collection:

```
Create a Postman collection for all the routes in the routes.js file
```

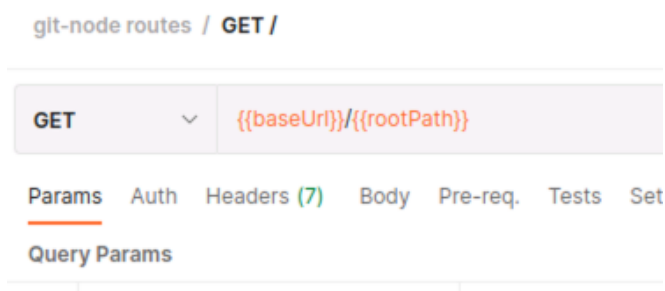
7. From the prompt in #6, the result is shown below:



8. Expand git-nodes routes to see the four endpoints:



9. At this point you may get a baseurl error, use Copilot to help you work through these errors.



10. In my case, it created a Global key value pair:

git-github Fork 0 Save

	VARIABLE	TYPE ⓘ	INITIAL VALUE ⓘ	CURRENT V ⓘ	Pe
☒	baseUrl	default	http://localhost:8000	http://localhost:8000	
☒	baseurl	default	http://localhost:8000	http://localhost:8000	
☒	rootPath	default	/	/	
☒	aboutUsPath	default	/aboutus	/aboutus	
☒	getEmployeesPath	default	/getemployees	/getemployees	
☒	addEmployeePath	default	/addemployee	/addemployee	
	Add a new variable				

11. It may be a challenge to get to this point, but in my case, I confirmed all three endpoints work, including /addemployee. The /getemployees route will not work unless a database was configured:

New Import Globals GET GET / GET GET /about POST POST /t

+ node-routes-four-endpoints

- GET GET /
- GET GET /aboutus
- GET GET /getemployees
- POST POST /addemployee

node-routes-four-endpoints / POST /addemployee

POST {{baseUrl}}/addemployee

Params Auth Headers (10) Body Pre-req. Tests Settings

x-www-form-urlencoded

☒	empName	Axle
☒	empPass	1234
	Key	Value

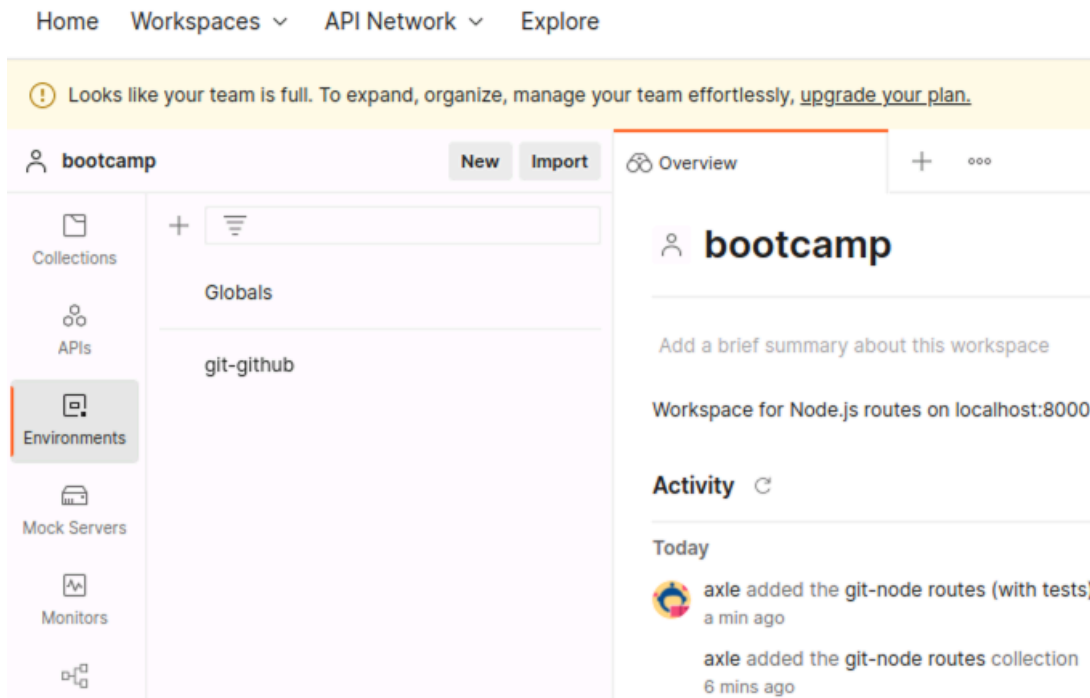
Body

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Created Axle"
3 }
```

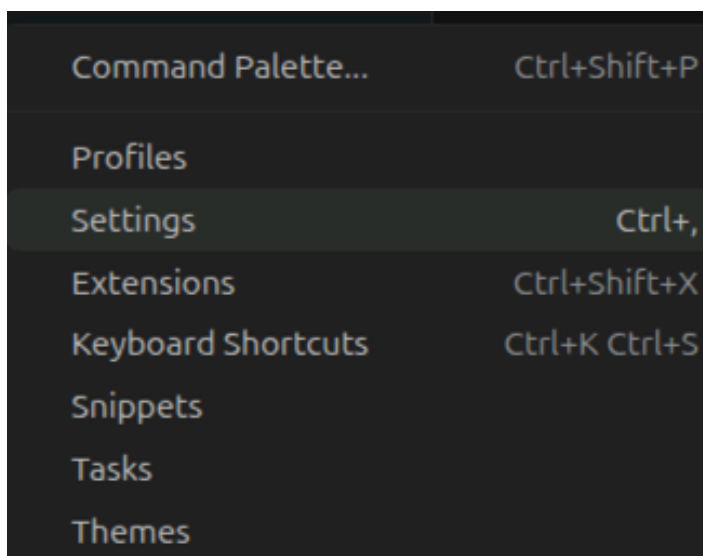
If you have gotten this far, you can attempt the sample Agentic Workflow in the next section of this bootcamp.

Appendix A – Local Postman Application Before Changes

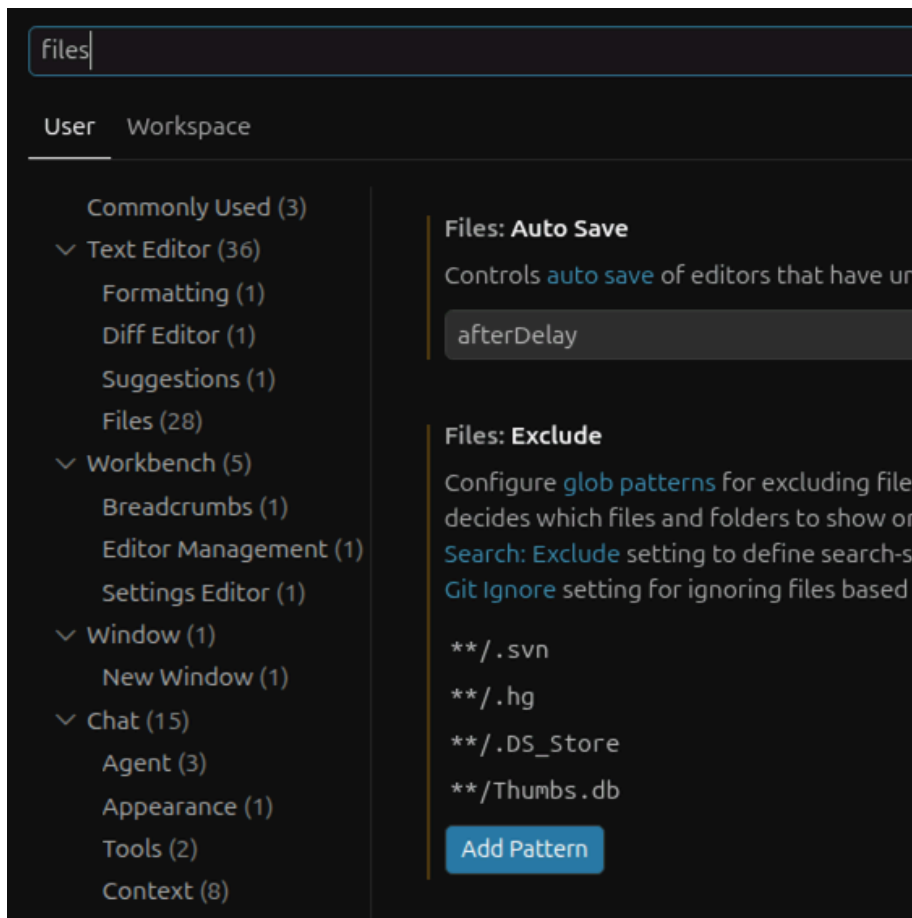


Appendix B – VS Code and Hidden Files

To enable VS Code to see hidden files, go to settings NOT Command Palette:



Start typing “files...” and you will see the Files:Exclude section:



Make sure that the type of file you want to see is NOT in this list.